RepairDesk — Complete Solution

> The whole project, every file. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck. All of this code runs.

The final layout

```
<repo folder>/
├── data/
│   └── sample/
│       └── messages.json
├── scripts/
│   └── intake_samples.py
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── concierge.py
│   │   ├── intake.py
│   │   └── router.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── prompts/
│   │   ├── concierge.txt
│   │   ├── intake.txt
│   │   └── router.txt
│   ├── services/
│   │   ├── __init__.py
│   │   └── shop_service.py
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── history.py
│   │   ├── logger.py
│   │   └── prompts.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   └── schemas.py
├── tests/
│   ├── integration/
```

```
│   │   └── test_live_agents.py
│   └── unit/
│       ├── test_config.py
│       ├── test_history.py
│       └── test_schemas.py
├── .env.example
├── main.py
└── requirements.txt
```

The project files you make once

These are the plain setup files. Copy them exactly — nothing here comes from another project.

`requirements.txt`

```
openai
python-dotenv
pydantic
```

`.env.example` (copy it to `.env` and put your real key in `.env`)

```
OPENAI_API_KEY=sk-your-key-here
LOG_LEVEL=INFO
```

`.gitignore`

```
.env
.venv/
__pycache__/
*.pyc
```

Every folder under `src/`, and `src/` itself, needs an **empty** `__init__.py` so Python treats it as a package: `src/__init__.py`, `src/agents/__init__.py`, `src/models/__init__.py`, `src/services/__init__.py`, `src/utils/__init__.py`. They hold nothing — the file just needs to exist.

`LICENSE` (MIT — fill in the year and your name)

```
MIT License

Copyright (c) 2026 <your name>
```

```
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Settings, errors, logging

```
# src/exceptions.py
# The project's named errors. MissingConfigError: a required setting is
# missing from .env.
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

```
# src/config.py
# Loads the settings from .env into a checked Config object. load_config()
# raises MissingConfigError when OPENAI_API_KEY is missing.
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
```

```
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

```
# src/utils/logger.py
# get_logger(name): one logging setup for every file — a timestamp, the
# file name and the level on every line.
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

Prompts and history helpers

```
# src/utils/prompts.py
# load_prompt(name): reads src/prompts/<name>.txt, so prompts live in text
# files instead of Python strings.

def load_prompt(name):
    with open("src/prompts" + "/" + name + ".txt") as prompt_file:
        return prompt_file.read().strip()
```

```
# src/utils/history.py
# Keeps a chat under the model's limit: estimate_tokens() guesses its size
# (about 4 characters per token), and trim_history() drops the oldest turns
# — never the system prompt.
```

```
def estimate_tokens(history):
    total = 0
    for message in history:
        total = total + len(message["content"])
    return total // 4

def trim_history(history, max_tokens):
    while estimate_tokens(history) > max_tokens and len(history) > 2:
        del history[1]
    return history
```

The one file that talks to OpenAI

```
# src/models/llm.py
# The only file that talks to the OpenAI SDK: create_client(), the startup
# key check, send_message(), and stream_message() for live replies.
import openai
from openai import OpenAI

MODEL = "gpt-4o-mini"

def create_client(config):
    return OpenAI(api_key=config.openai_api_key)

def check_api_key_at_startup(client):
    try:
        client.chat.completions.create(
            model=MODEL,
            messages=[{"role": "user", "content": "ping"}],
            max_tokens=1,
        )
    except openai.AuthenticationError:
        return False
    return True

def send_message(client, history, user_input):
    history.append({"role": "user", "content": user_input})
    response = client.chat.completions.create(model=MODEL, messages=history)
    reply = response.choices[0].message.content
    history.append({"role": "assistant", "content": reply})
    return reply

def stream_message(client, history, user_input):
    history.append({"role": "user", "content": user_input})
```

```
    stream = client.chat.completions.create(
        model=MODEL, messages=history, stream=True
    )
    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="", flush=True)
            full_reply = full_reply + piece
    print()
    history.append({"role": "assistant", "content": full_reply})
    return full_reply
```

The fixed shapes

```
# src/schemas.py
# The shapes the model must answer in. RepairTicket: one repair ticket.
# RouteDecision: which agent gets a message, and why.
from typing import Literal
from pydantic import BaseModel

class RepairTicket(BaseModel):
    customer_name: str | None
    device: Literal["phone", "laptop", "tablet", "other"]
    urgency: Literal["low", "medium", "high"]
    summary: str

class RouteDecision(BaseModel):
    agent: Literal["concierge", "intake"]
    reason: str
```

The three agents

```
# src/agents/intake.py
# The Intake agent: turns one problem message into a RepairTicket, or
# returns None when the model's answer can't be used.
from pydantic import ValidationError

from src.models.llm import MODEL
from src.schemas import RepairTicket
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt
```

```
def intake_agent(client, message):
    logger = get_logger(__name__)
    messages = [
        {"role": "system", "content": load_prompt("intake")},
        {"role": "user", "content": message},
    ]
    try:
        completion = client.beta.chat.completions.parse(
            model=MODEL, messages=messages, response_format=RepairTicket
        )
    except ValidationError as e:
        logger.warning(f"Intake output did not match RepairTicket: {e}")
        return None
    reply = completion.choices[0].message
    if reply.refusal:
        logger.warning(f"Intake agent refused: {reply.refusal}")
        return None
    return reply.parsed
```

```
# src/agents/concierge.py
# The Concierge agent: start_history() begins a chat with its system
# prompt, and concierge_agent() trims the chat and streams the reply.

from src.models.llm import stream_message
from src.utils.history import trim_history
from src.utils.prompts import load_prompt

def start_history():
    return [{"role": "system", "content": load_prompt("concierge")}]

def concierge_agent(client, history, message):
    trim_history(history, 3000)
    return stream_message(client, history, message)
```

```
# src/agents/router.py
# The router: decide() asks the model which agent should answer a message,
# and falls back to the Concierge when the answer can't be used.
from pydantic import ValidationError

from src.models.llm import MODEL
from src.schemas import RouteDecision
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt
```

```
def decide(client, message):
    logger = get_logger(__name__)
    messages = [
        {"role": "system", "content": load_prompt("router")},
        {"role": "user", "content": message},
    ]
    decision = RouteDecision(
        agent="concierge", reason="router could not decide"
    )
    try:
        completion = client.beta.chat.completions.parse(
            model=MODEL, messages=messages, response_format=RouteDecision
        )
        reply = completion.choices[0].message
        if reply.parsed is not None:
            decision = reply.parsed
    except ValidationError as e:
        logger.warning(f"Router output did not match RouteDecision: {e}")
    logger.info(f"routed to {decision.agent} ({decision.reason}): {message}")
    return decision
```

The service that ties it together

```
# src/services/shop_service.py
# The app's flow, apart from the keyboard: handle_message() asks the router,
# then calls the chosen agent. print_ticket() shows a ticket.

from src.agents.concierge import concierge_agent
from src.agents.intake import intake_agent
from src.agents.router import decide
from src.utils.logger import get_logger

def print_ticket(ticket):
    name = ticket.customer_name
    if name is None:
        name = "(not given)"
    print("Repair ticket created:")
    print(f"  customer: {name}")
    print(f"  device:   {ticket.device}")
    print(f"  urgency:  {ticket.urgency}")
    print(f"  summary:  {ticket.summary}")

def handle_message(client, history, message):
    logger = get_logger(__name__)
    decision = decide(client, message)
```

```
    if decision.agent == "intake":
        ticket = intake_agent(client, message)
        if ticket is None:
            print("Sorry, I couldn't turn that into a repair ticket — please")
            print("describe the problem again in a little more detail.")
        else:
            logger.info(f"ticket: {ticket.device} / {ticket.urgency}")
            print_ticket(ticket)
    else:
        print("Concierge: ", end="", flush=True)
        concierge_agent(client, history, message)
```

The prompts

`src/prompts/concierge.txt`

```
You are the RepairDesk concierge. Be friendly and brief.
Answer general questions about the repair shop: hours, prices, services.
```

`src/prompts/intake.txt`

```
You take repair intake. Turn the customer message into a repair ticket.
Fill customer_name only if the customer states their name - otherwise leave
it null, never guess one. Pick device from the message. Use urgency "high"
only if the customer says it is urgent or needed today.
```

`src/prompts/router.txt`

```
You route a message to one agent. Use "intake" if the customer describes
a device problem to repair. Use "concierge" for general questions, prices,
hours or small talk. Give a one-sentence reason.
```

The sample data, the runner, and main.py

`data/sample/messages.json`

```
{
  "intake": [
    "Hi, I'm Sara Khan. My phone screen is cracked, please fix it today.",
    "my laptop won't turn on, tried charging overnight, still nothing",
    "my tablet is really slow and freezes a lot, no rush"
```

```
  ],
  "routing": [
    {"message": "hi there, how are you today?", "expected": "concierge"},
    {"message": "what are your repair hours?", "expected": "concierge"},
    {"message": "my phone screen is cracked, fix it", "expected": "intake"},
    {"message": "my laptop won't turn on at all", "expected": "intake"}
  ]
}
```

```
# scripts/intake_samples.py
# Runs the Intake agent on every sample message and prints each ticket.
# Run from the repo root: python -m scripts.intake_samples
import json

from src.agents.intake import intake_agent
from src.config import load_config
from src.models.llm import create_client

SAMPLES_PATH = "data/sample/messages.json"

def main():
    config = load_config()
    client = create_client(config)
    with open(SAMPLES_PATH) as samples_file:
        samples = json.loads(samples_file.read())
    for message in samples["intake"]:
        print(f"\nMessage: {message}")
        ticket = intake_agent(client, message)
        if ticket is None:
            print("No ticket — the Intake agent couldn't extract one.")
        else:
            print(f"Ticket:  {ticket}")

if __name__ == "__main__":
    main()
```

```
# main.py
# The file you run: python main.py
# Startup, the input loop and the error messages — every message goes to
# handle_message(), which picks the agent.
import sys

import openai

from src.agents.concierge import start_history
```

```
from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import check_api_key_at_startup, create_client
from src.services.shop_service import handle_message
from src.utils.logger import get_logger

def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    client = create_client(config)
    if not check_api_key_at_startup(client):
        logger.error("Authentication failed — check OPENAI_API_KEY.")
        print("Login failed — check OPENAI_API_KEY in your .env file.")
        sys.exit(1)

    history = start_history()
    print("RepairDesk AI — type a message, or /quit to exit.")

    while True:
        message = input("You: ").strip()
        if message == "/quit":
            break
        if message == "":
            continue
        try:
            handle_message(client, history, message)
        except openai.AuthenticationError:
            logger.error("API key stopped working mid-session.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)
        except openai.RateLimitError:
            logger.warning("Rate limited after the client's retries.")
            print("We're busy right now — wait a moment and try again.")
        except openai.BadRequestError as e:
            logger.warning(f"Request rejected (likely too long): {e}")
            if history[-1]["role"] == "user":
                history.pop()
            print("That message was too long to handle — please shorten it.")

if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Checks the settings code with no API call. Each case prints OK or WRONG.
# It only changes settings inside this test run — your .env is never touched.
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "REPAIRDESK_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting():
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

```
# tests/unit/test_history.py
# Checks the history helpers. Each case prints OK or WRONG. No API key.
# Run from the repo root: python -m tests.unit.test_history
from src.utils.history import estimate_tokens, trim_history

def test_estimate_tokens():
    history = [
        {"role": "system", "content": "a" * 20},
        {"role": "user", "content": "b" * 20},
    ]
    result = "OK"
    if estimate_tokens(history) != 10:
        result = "WRONG"
    print(f"case 1 (40 chars = 10 tokens) -> {result}")

def test_short_history_untouched():
    history = [
        {"role": "system", "content": "be nice"},
        {"role": "user", "content": "hi"},
    ]
    trim_history(history, 100)
    result = "OK"
    if len(history) != 2:
        result = "WRONG"
    print(f"case 2 (short chat not trimmed) -> {result}")

def test_long_history_trimmed():
    history = [{"role": "system", "content": "be nice"}]
    for number in range(1, 6):
        history.append({"role": "user", "content": str(number) * 400})
    trim_history(history, 250)
    result = "OK"
    if history[0]["role"] != "system":
        result = "WRONG"
    if history[-1]["content"] != "5" * 400:
        result = "WRONG"
    print(f"case 3 (long chat trimmed)     -> {result}")

if __name__ == "__main__":
    test_estimate_tokens()
    test_short_history_untouched()
    test_long_history_trimmed()
```

```
# tests/unit/test_schemas.py
# Checks the shapes accept good data and reject bad data. Each case prints
# OK or WRONG. No API key needed.
```

```
# Run from the repo root: python -m tests.unit.test_schemas
from pydantic import ValidationError

from src.schemas import RepairTicket, RouteDecision

def test_ticket_without_a_name():
    ticket = RepairTicket(
        customer_name=None, device="laptop", urgency="medium",
        summary="Laptop won't turn on.",
    )
    result = "OK"
    if ticket.customer_name is not None:
        result = "WRONG"
    print(f"case 1 (no name allowed) -> {result}")

def test_ticket_rejects_unknown_device():
    result = "WRONG"
    try:
        RepairTicket(customer_name="Sara", device="fridge",
                     urgency="low", summary="x")
    except ValidationError:
        result = "OK"
    print(f"case 2 (bad device)      -> {result}")

def test_route_rejects_unknown_agent():
    result = "WRONG"
    try:
        RouteDecision(agent="billing", reason="looks like billing")
    except ValidationError:
        result = "OK"
    print(f"case 3 (bad agent name)  -> {result}")

if __name__ == "__main__":
    test_ticket_without_a_name()
    test_ticket_rejects_unknown_device()
    test_route_rejects_unknown_agent()
```

```
# tests/integration/test_live_agents.py
# Checks the app against the real OpenAI API. Each case prints OK or WRONG.
# Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents
import json

from openai import OpenAI

from src.agents.intake import intake_agent
```

```
from src.agents.router import decide
from src.config import load_config
from src.models.llm import (
    check_api_key_at_startup,
    create_client,
    send_message,
)

SAMPLES_PATH = "data/sample/messages.json"

def test_first_reply(client):
    reply = send_message(client, [], "Hello")
    result = "OK"
    if not reply:
        result = "WRONG"
    print(f"case 1 (first reply) -> {result}")

def test_bad_key():
    bad_client = OpenAI(api_key="sk-fake-key-123")
    result = "OK"
    if check_api_key_at_startup(bad_client):
        result = "WRONG"
    print(f"case 2 (bad key)     -> {result}")

def load_samples():
    with open(SAMPLES_PATH) as samples_file:
        return json.loads(samples_file.read())

def test_every_sample_gets_a_ticket(client, samples):
    result = "OK"
    for message in samples["intake"]:
        if intake_agent(client, message) is None:
            result = "WRONG"
    print(f"case 3 (tickets)     -> {result}")

def test_no_invented_name(client, samples):
    ticket = intake_agent(client, samples["intake"][1])
    result = "OK"
    if ticket is None or ticket.customer_name is not None:
        result = "WRONG"
    print(f"case 4 (no name)     -> {result}")

def test_routing(client, samples):
    result = "OK"
    for case in samples["routing"]:
        if decide(client, case["message"]).agent != case["expected"]:
            result = "WRONG"
```

```
    print(f"case 5 (routing)     -> {result}")

if __name__ == "__main__":
    config = load_config()
    client = create_client(config)
    samples = load_samples()
    test_first_reply(client)
    test_bad_key()
    test_every_sample_gets_a_ticket(client, samples)
    test_no_invented_name(client, samples)
    test_routing(client, samples)
```

Expected unit output (all `OK`):

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (40 chars = 10 tokens) -> OK
case 2 (short chat not trimmed) -> OK
case 3 (long chat trimmed)     -> OK
```

```
case 1 (no name allowed) -> OK
case 2 (bad device)      -> OK
case 3 (bad agent name)  -> OK
```

That's the whole project. If your version routes a question to the Concierge, turns a problem into a correctly-shaped ticket, leaves an unstated name empty, and never crashes on a bad key or a long message — you've built it.